Lecture L19

*Blind review — please score this lecture on row **L19** in the Excel form (1–10 per dimension).*

Learning Objectives

By the end of this lecture, you will be able to:

1. *Remember* the definitions and conditions for 1NF, 2NF, 3NF, and BCNF using precise terminology.
2. *Understand* how functional dependencies and candidate keys determine whether a schema violates a normal form.
3. *Apply* decomposition techniques to convert a schema into 3NF or BCNF, ensuring lossless join.
4. *Analyze* a given schema for data redundancy and anomalies, identifying which normal form it satisfies (if any).
5. *Evaluate* trade-offs between full normalization, dependency preservation, and query performance in real-world design.

Prerequisites: Functional Dependencies and Keys

Before discussing normalization, we establish key concepts.

A **functional dependency (FD)** $X \to Y$ holds in a relation if, for any two tuples, equality on all attributes in X implies equality on all attributes in Y. Formally, if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$. For example, SSN $\to$ Name means no two individuals with the same SSN can have different names.

A **candidate key** is a minimal set of attributes that functionally determines all other attributes in the relation. Minimality implies that no proper subset of the key is also a superkey.

> **Quick Check**: In the relation `Enroll(StudentID, Course, Grade)` with FDs StudentID $\to$ StudentName and $\{\text{StudentID}, \text{Course}\} \to$ Grade, is $\{\text{StudentID}, \text{Course}\}$ a candidate key?
> *(Answer: Yes — it determines all attributes, and no smaller subset does.)*

Functional dependencies and candidate keys are foundational to normalization. If these concepts are unfamiliar, pause to review them now.

Motivation: The Problems of Unnormalized Data — A Paradox in Plain Sight

Imagine a database that *seems* to work perfectly: queries return correct results, inserts succeed, and reports generate on time. Yet, one day, a single update turns a consistent dataset into a contradictory mess — and no constraint was violated.

This is not a software bug. It's a *design paradox*: a schema can be logically inconsistent even when every individual operation appears valid.

Consider a university tracking faculty assignments:

Instructor	Course	Department	Building
Dr. Lee	CS101	Computer Science	Tech Hall
Dr. Lee	CS202	Computer Science	Tech Hall
Dr. Patel	MATH101	Mathematics	Science Wing

Instructor	Course	Department	Building
Dr. Patel	MATH202	Mathematics	Science Wing

Suppose Dr. Lee changes departments. You update two rows — but what if a third copy exists in a forgotten backup table? Or a script updates only one row due to a typo? The database allows this inconsistency because it cannot *enforce* that Dr. Lee has exactly one department.

This is not a failure of data entry — it's a failure of *information encoding*. The schema treats "Dr. Lee → Computer Science" as repeated data, not as a fact.

> **Try this yourself**: Sketch a simple constraint (e.g., `CHECK`, `UNIQUE`, `FOREIGN KEY`) that would prevent Dr. Lee from having two different departments.
> *(Spoiler: You can't — not without restructuring the schema.)*

Now try this: design a schema where such inconsistency is *impossible*, even with buggy code. What would need to change?

> **Interactive Prompt**: Write down the tables you'd create and the keys/FDs that would enforce one department per instructor. Come back to this after 3NF — your answer should match the decomposition we derive.

Normalization resolves this by transforming *redundant data* into *enforceable facts*. Each normal form is a rule that turns implicit assumptions into explicit, checkable constraints.

Industry Relevance: In 2020, a major e-commerce platform experienced a pricing inconsistency across product listings due to denormalized inventory tables. A discount applied to a category was missed in 12% of product rows because category-level rules were embedded in product records instead of being stored once in a separate category table. The bug went undetected for weeks, leading to revenue loss and customer complaints. This is a classic 2NF/3NF violation: category attributes depended on a partial key and were redundantly repeated across products.

First Normal Form (1NF): The Atomicity Contract

Definition: A relation is in **First Normal Form (1NF)** if: 1. Each attribute contains only atomic (indivisible) values. 2. There are no repeating groups within a tuple.

> **Analogy**: Think of 1NF as the *periodic table of database design*. Just as chemical compounds must be broken into atoms to predict reactions, data must be decomposed into atomic values to predict behavior under operations. You can't reliably query a "compound" value — you need its constituent elements.

Key Insight: Atomicity is *usage-dependent*. A string `"CS101, CS202"` is atomic if treated as a course bundle. But if queries filter by individual course, it's a molecule — and must be split.

Worked Example
Suppose:

Ename	Projects
Alice	"HR, Payroll"
Bob	"Inventory"

This violates 1NF — `Projects` is a container, not a scalar.

Fix: Promote each project to its own row:

Ename	Project
Alice	HR
Alice	Payroll
Bob	Inventory

Now the schema treats projects as first-class entities — a prerequisite for meaningful queries.

> **Test Yourself**: Which violate 1NF?
> A) A column storing JSON `{"city": "Boston", "zip": "02115"}`
> B) A column with NULL values
> C) A column storing a list: "CS101, CS202"
> *(Answers: A and C — both embed structure without relational decomposition. B is fine: NULL is a valid placeholder in 1NF.)*

> **Try this yourself**: Suppose you're building a food delivery app. You store a user's favorite cuisines as a comma-separated string: `"Italian, Mexican, Thai"`. How would you normalize this into 1NF? What new table and key would you introduce?

Industry Application: Modern applications often ingest semi-structured data (e.g., JSON logs, user preferences). A common anti-pattern is storing JSON blobs in a single column and parsing them in application code. For example, a ride-sharing app might store driver availability as:

```
{ "days": ["Mon", "Wed", "Fri"], "hours": "9AM-5PM" }
```

While convenient, this prevents efficient querying (e.g., "find all drivers available on Wednesdays") and violates 1NF. Best practice, as used by companies like Uber and Lyft, is to normalize such data into junction tables (e.g., `DriverAvailability(driver_id, day, start_time, end_time)`), enabling indexing and constraint enforcement.

Limitation: 1NF ensures clean cells, but not clean logic. Alice still appears twice — redundancy remains. We now move from *structure* to *meaning*.

Second Normal Form (2NF): The Key Is the Whole Key

We define 2NF only for relations in 1NF.

Definition: A relation is in **Second Normal Form (2NF)** if: - It is in 1NF, and - Every non-prime attribute is fully functionally dependent on every candidate key.

> **Analogy**: Imagine a composite key as a *combination lock*. If you can open it with only part of the code (e.g., just the first number), the lock is broken. Similarly, if a non-key attribute depends on only part of a composite key, the schema is vulnerable.

Key Terms: - **Prime attribute**: Part of any candidate key. - **Non-prime attribute**: Not part of any candidate key. - **Partial dependency**: $X \to Y$ where X is a proper subset of a candidate key.

Worked Example
`Registration(StudentID, CourseID, CourseName)`
- FD: CourseID → CourseName - Candidate key: {StudentID, CourseID}

CourseName depends only on CourseID — a partial dependency.

This is like storing the color of a car in every rental record instead of in the car catalog. The dependency is real, but its *location* is wrong.

Fix: Decompose: - `Enrollment(StudentID, CourseID)` - `Course(CourseID, CourseName)`

Now, `CourseName` depends on the full key of its own table.

> **Test Yourself**: In `Order(ItemID, SupplierID, SupplierCity)` with key {ItemID, SupplierID} and SupplierID → SupplierCity, is `SupplierCity` fully dependent on the key?
> *(Answer: No — it depends only on `SupplierID`, a subset of the key. This violates 2NF.)*

> **Try this yourself**: Suppose you have a table `Flight(PlaneType, Airline, AirlineHQ)` with key {PlaneType, Airline} and FD Airline → AirlineHQ. Identify the partial dependency and sketch a 2NF decomposition.

Why It Matters: 2NF eliminates redundancy caused by *misplaced* dependencies — those that belong to an entity different from the one being described. Without this, every update to `AirlineHQ` would require scanning all flights operated by that airline.

Third Normal Form (3NF): No Transitive Dependencies — Only Direct Causes

Definition: A relation is in **Third Normal Form (3NF)** if: - It is in 2NF, and - No non-prime attribute is transitively dependent on any candidate key.

That is, if $K \to A$ and $A \to B$, then B must not be non-prime unless $A \to K$.

> **Analogy**: Think of 3NF like a **supply chain audit**. If a factory (the key) produces a car, and the car determines the tire brand, but the tire brand determines the rubber supplier — you wouldn't store the rubber supplier directly in the factory record. That's a transitive dependency: factory → car → tire → supplier. You only store direct suppliers. Similarly, in 3NF, we only allow attributes that depend *directly* on the key.

Worked Example
`Faculty(Instructor, Department, Building)`
- FDs: Instructor → Department, Department → Building - Candidate key: Instructor

Here, Instructor → Department → Building, so `Building` is transitively dependent on the key.

If the department moves buildings, you must update every faculty record in that department — and risk inconsistency.

Fix: Decompose: - `Faculty(Instructor, Department)` - `DeptLocation(Department, Building)`

Now, `Building` depends directly on `Department`, not through a chain.

> **Try this yourself**: In a table `Patient(SSN, DoctorID, DoctorOffice)`, with SSN → DoctorID and DoctorID → DoctorOffice, is this in 3NF? If not, decompose it.

Why It Matters: 3NF prevents update anomalies where changing one fact (e.g., office location) forces changes in many unrelated records. It ensures that each fact has a single, authoritative home.

Boyce-Codd Normal Form (BCNF): Every Dependency Has a Key on the Left

Definition: A relation is in **BCNF** if, for every non-trivial FD $X \to Y$, X is a superkey.

This is stronger than 3NF — it removes even some dependencies involving prime attributes.

> **Analogy**: BCNF is like a **constitutional government** — every rule (functional dependency) must be issued by a legitimate authority (a superkey). If a non-authority (non-superkey) can dictate a fact, the system is unstable.

Worked Example
`Teaching(Professor, Course, Assistant)`
- FDs: Professor → Course, Course → Assistant - Candidate keys: {Professor, Assistant}, {Course, Professor}

Check Course → Assistant: Is `Course` a superkey? No — multiple professors might teach the same course, but here `Course` alone determines `Assistant`, violating BCNF.

Suppose CS101 always uses the same TA, regardless of who teaches it. But if two professors teach CS101, they must share the same assistant — a constraint the schema enforces, but one that may not reflect reality.

Fix: Decompose based on the violating FD: - `CourseAssistant(Course, Assistant)` - `ProfCourse(Professor, Course)`

Now, every FD has a superkey on the left.

> **Try this yourself**: In `Enroll(StudentID, Course, Instructor)` with FDs Course → Instructor and key {StudentID, Course}, does this violate BCNF? Why? How would you decompose it?

Key Insight: BCNF eliminates redundancy from *any* non-superkey determinant. But — and this is critical — it may not preserve all functional dependencies. Sometimes, you must choose between full normalization and maintaining all constraints.

> **Tension Point**: Can every schema be decomposed into BCNF while preserving dependencies and ensuring lossless join?
> *(Spoiler: No. There are schemas where you must sacrifice one. This is why 3NF is still used in practice.)*

Summary: The Normalization Journey

We've moved from: - **1NF**: Clean cells (atomic values) - **2NF**: No partial dependencies (whole key matters) - **3NF**: No transitive dependencies (only direct causes) - **BCNF**: Every determinant is a superkey (only authorities decide)

Each step eliminates a new kind of redundancy by promoting facts to their proper homes.

> **Final Challenge**: Revisit the faculty assignment table from the beginning:

Instructor	Course	Department	Building
Dr. Lee	CS101	Computer Science	Tech Hall
Dr. Lee	CS202	Computer Science	Tech Hall

Apply full normalization. Identify all FDs, candidate keys, and decompose into BCNF. Does your solution prevent the department inconsistency? Does it preserve all dependencies?

This is not just theory — it's the difference between a database that breaks silently and one that *cannot* break.